

UNIVERSITÉ INTERNATIONALE DE RABAT
Filière : Big Data & Intelligence Artificielle – 4^{ème} Année

RAPPORT DE STAGE

Conception et développement d'un système intelligent de génération automatique de Notes DSI

*Automatisation par agents IA multi-modèles, classification de documents
par réseau de neurones convolutif (CNN) et intégration Gmail*

Présenté par :
Manal SAWLI

Encadré par :
Basma GUERMAH – UIR
Imane AMELI – DSI

Organisme d'accueil :
Ministère de l'Agriculture, de la Pêche Maritime, du
Développement Rural et des Eaux et Forêts
Direction des Systèmes d'Information (DSI)

Période de stage :
1 juin 2026 – 5 juillet 2026

Année universitaire 2025 – 2026

Remerciements

Je tiens à exprimer ma sincère gratitude à **Madame Imane AMELI**, mon encadrante au sein de la Direction des Systèmes d'Information, ainsi qu'à **Madame Basma GUERMAH**, mon encadrant à l'Université Internationale de Rabat, pour leur disponibilité, leurs conseils précieux et leur accompagnement tout au long de ce stage. Je remercie également l'ensemble du personnel de la DSI pour leur accueil chaleureux, ainsi que le corps professoral de l'UIR pour la formation qui m'a préparé à ce projet.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage au sein de la Direction des Systèmes d'Information (DSI) du Ministère de l'Agriculture, de la Pêche Maritime, visant à automatiser la rédaction des notes administratives internes (« Notes DSI ») à partir des emails Gmail de la direction. La solution combine une intégration avec l'API Gmail, un modèle de classification d'images par CNN (ResNet18) pour identifier le type de document joint, un système multi-agents (CrewAI) s'appuyant sur un LLM local (Mistral via Ollama) pour extraire et reformuler les informations, ainsi qu'un module de génération de documents Word fidèle à la charte graphique du Ministère. Accessible via une interface Streamlit, l'application permet de générer en un clic une note DSI téléchargeable. Le modèle CNN atteint une précision supérieure à 95 % en validation sur les quatre classes de documents considérées.

Mots-clés : Intelligence Artificielle, CNN, ResNet18, Agents IA, CrewAI, LLM, Gmail API, Automatisation, Streamlit, OLLAMA, MISTRAL.

Abstract

This report presents an internship carried out at the Information Systems Directorate (DSI) of the Moroccan Ministry of Agriculture, Maritime Fisheries, Rural Development, aiming to automate the drafting of internal administrative notes (“Notes DSI”) from the department’s Gmail emails. The solution combines Gmail API integration, a CNN-based (ResNet18) image classification model to identify the attached document type, a multi-agent system (CrewAI) relying on a local LLM (Mistral via Ollama) to extract and rephrase information, and a Word document generation module matching the Ministry’s official letterhead. Accessible through a Streamlit interface, the application generates a downloadable DSI note with a single click. The CNN model achieves over 95 % validation accuracy across the four document classes considered.

Keywords: Artificial Intelligence, CNN, ResNet18, IA Agents, CrewAI, LLM, Gmail API, Automatisations, Streamlit, OLLAMA, MISTRAL..

Contents

Remerciements	i
Résumé	ii
Abstract	iii
List of Figures	vii
1 Contexte général du stage	1
1.1 Introduction	1
1.2 Présentation de l'organisme d'accueil	1
1.3 Problématique	1
1.4 Objectifs du stage	2
1.5 Méthodologie de travail	3
1.6 Conclusion	3
2 Étude de l'existant et analyse des besoins	4
2.1 Introduction	4
2.2 Processus manuel existant	4
2.3 Limites du processus actuel	5
2.4 Analyse des besoins fonctionnels	5
2.5 Analyse des besoins non fonctionnels	6
2.6 Choix technologiques envisagés	7
2.7 Conclusion	7
3 Conception de la solution	8
3.1 Introduction	8
3.2 Architecture globale	8
3.3 Conception du module de classification (CNN)	9
3.3.1 Choix de l'architecture	9
3.3.2 Classes de documents	9
3.3.3 Stratégie de constitution du jeu de données	10
3.4 Conception du système multi-agents (CrewAI)	10

3.4.1	Principe général	10
3.4.2	Les quatre agents	10
3.4.3	Choix du modèle de langage	11
3.4.4	Flux d'exécution des tâches	11
3.5	Conception du module de génération de document	11
3.6	Conception de l'interface utilisateur	12
3.7	Conclusion	12
4	Réalisation	13
4.1	Introduction	13
4.2	Environnement de développement	13
4.3	Entraînement du modèle de classification (CNN)	15
4.3.1	Génération du jeu de données synthétique	15
4.3.2	Architecture du modèle	16
4.3.3	Boucle d'entraînement	17
4.3.4	Résultats de l'entraînement	17
4.3.5	Exposition du modèle comme outil CrewAI	19
4.4	Système multi-agents avec CrewAI	19
4.4.1	Définition des agents	20
4.4.2	Définition des tâches	20
4.4.3	Choix de conception : classification affichée hors LLM	21
4.5	Intégration avec l'API Gmail	21
4.5.1	Authentification OAuth 2.0	21
4.5.2	Extraction du corps de l'email et des pièces jointes	22
4.6	Génération du document Word	22
4.6.1	Reproduction fidèle de la charte graphique	23
4.6.2	Structuration du tableau de la note	23
4.6.3	Robustesse du parsing JSON	24
4.7	Conclusion	24
5	Tests et résultats	25
5.1	Introduction	25
5.2	Interface d'accueil	25
5.3	Chargement et sélection d'un email	25
5.4	Génération de la note	27
5.5	Aperçu et validation du résultat	27
5.6	Téléchargement et document final	29
5.7	Analyse des résultats	30
5.7.1	Performance du modèle de classification	30

5.7.2	Pertinence du contenu généré par les agents	31
5.7.3	Fiabilité de l'architecture retenue	31
5.7.4	Limites observées	31
5.8	Conclusion	32
Conclusion générale et perspectives		33
	Synthèse du travail réalisé	33
	Apports personnels du stage	33
	Perspectives d'amélioration	34

List of Figures

1.1	En-tête officiel de la Direction des Systèmes d'Information (DSI)	1
3.1	Architecture globale de la solution	9
4.1	Arborescence du projet dans Visual Studio Code	14
4.2	Dépendances principales du projet (requirements.txt)	15
4.3	Génération d'une image synthétique de type "Facture"	16
4.4	Construction du modèle ResNet18 adapté	16
4.5	Boucle d'entraînement d'une époque	17
4.6	Courbe de perte (entraînement et validation) sur 6 époques	18
4.7	Matrice de confusion du modèle CNN sur l'ensemble de validation . . .	18
4.8	Chargement paresseux du modèle et fonction de classification	19
4.9	Définition de l'agent Classificateur	20
4.10	Extrait de la tâche d'extraction d'informations	21
4.11	Authentification et création du service Gmail	22
4.12	Aplatissage récursif des parties MIME d'un email	22
4.13	Ajout manuel des bordures de cellule en XML (OOXML)	23
4.14	Construction des lignes du tableau de la note	24
5.1	Page d'accueil de l'application Streamlit	25
5.2	Liste des emails chargés depuis Gmail, avec pièces jointes détectées . .	26
5.3	Aperçu de l'email sélectionné avant génération	27
5.4	Indicateur de traitement pendant l'exécution du pipeline multi-agents .	27
5.5	Aperçu de la note DSI générée avant téléchargement	28
5.6	Téléchargement du fichier Note_DSI.docx	29
5.7	Note DSI finale générée, ouverte dans Microsoft Word	30

Chapter 1

Contexte général du stage

1.1 Introduction

Ce premier chapitre a pour objectif de situer le cadre général dans lequel s'est déroulé le stage. Nous y présentons successivement l'organisme d'accueil et sa direction des systèmes d'information, avant d'exposer la problématique ayant motivé ce projet ainsi que les objectifs fixés pour ce stage.

1.2 Présentation de l'organisme d'accueil

Le stage s'est déroulé au sein de la **Direction des Systèmes d'Information (DSI)** du Ministère de l'Agriculture, de la Pêche Maritime, du Développement Rural et des Eaux et Forêts, structure chargée de la gouvernance, du développement et de la maintenance du système d'information du Ministère, ainsi que de l'accompagnement digital de ses différentes directions.



Figure 1.1: En-tête officiel de la Direction des Systèmes d'Information (DSI)

1.3 Problématique

Au sein de la DSI, une part importante de la communication professionnelle transite par la messagerie électronique institutionnelle (Gmail). De nombreux échanges avec des partenaires, prestataires, organismes publics ou usagers nécessitent d'être formalisés sous la forme d'une « **Note DSI** » : un document Word structuré, respectant une charte graphique précise (en-tête ministériel, tableau normé avec les champs Cadre, Objet, Participants, Descriptif et Prochaine Action), destiné à informer la hiérarchie et à tracer les décisions prises.

Cette tâche, bien que simple en apparence, présente plusieurs limites lorsqu'elle est réalisée manuellement :

- elle est **chronophage**, chaque note nécessitant la relecture de l'email, l'identification des informations utiles et leur reformulation ;
- elle est **répétitive** pour les agents traitant un grand nombre de correspondances similaires ;
- elle est sujette à des **incohérences de forme** d'un agent à l'autre (mise en page, niveau de détail, style rédactionnel) ;
- elle nécessite une **identification préalable du type de document** joint (facture, courrier officiel, formulaire, etc.), généralement faite visuellement par l'agent.

La problématique centrale de ce stage peut donc être formulée ainsi : *comment automatiser, à partir d'un email et de ses éventuelles pièces jointes, la génération d'une Note DSI complète, fidèle à la charte graphique du Ministère, tout en conservant un contrôle humain sur le résultat produit ?*

1.4 Objectifs du stage

Pour répondre à cette problématique, les objectifs suivants ont été fixés en début de stage :

1. Étudier le processus manuel existant de rédaction des notes DSI et identifier les informations structurantes à extraire d'un email.
2. Concevoir une architecture logicielle permettant de connecter la messagerie Gmail de la DSI, de lire les emails et leurs pièces jointes.
3. Développer un modèle de **classification automatique de documents** (facture, courrier officiel, formulaire, note interne) basé sur un réseau de neurones convolutif (CNN), afin d'identifier la nature des pièces jointes reçues.
4. Mettre en place un **système multi-agents** capable d'extraire, structurer et reformuler en français administratif les informations utiles contenues dans l'email.
5. Générer automatiquement un **document Word (.docx)** reproduisant fidèlement la charte graphique officielle des notes DSI (logos, tableau, mise en page).

6. Développer une **interface utilisateur simple et intuitive** permettant à un agent de piloter l'ensemble du processus, de la sélection de l'email jusqu'au téléchargement de la note.

1.5 Méthodologie de travail

Le développement du projet a suivi une démarche itérative, structurée en plusieurs phases : analyse du besoin et de l'existant, conception de l'architecture, développement des modules indépendants (classification, extraction, génération de document), intégration, puis tests et validation. Cette approche a permis de valider progressivement chaque brique technologique avant son intégration dans le système global, réduisant ainsi les risques liés à la complexité de l'ensemble.

1.6 Conclusion

Ce chapitre a permis de poser le cadre général du stage : l'organisme d'accueil et sa direction d'affectation, ainsi que la problématique et les objectifs qui ont guidé l'ensemble du travail réalisé. Le chapitre suivant s'attache à détailler plus précisément le processus existant et les besoins fonctionnels et non fonctionnels identifiés.

Chapter 2

Étude de l'existant et analyse des besoins

2.1 Introduction

Avant d'entamer la conception de la solution, il est indispensable de bien comprendre le processus métier existant ainsi que les besoins réels des utilisateurs finaux. Ce chapitre présente le processus manuel actuellement en place au sein de la DSI, en identifie les limites, puis formalise les besoins fonctionnels et non fonctionnels du système à développer.

2.2 Processus manuel existant

Le processus actuel de rédaction d'une note DSI suit les étapes suivantes :

1. L'agent reçoit un email sur sa messagerie Gmail professionnelle, éventuellement accompagné d'une ou plusieurs pièces jointes (factures, courriers, formulaires, etc.).
2. Il ouvre l'email, en lit le contenu et identifie manuellement les informations clés : objet de la demande, expéditeur(s)/participants, contenu détaillé, action à entreprendre.
3. Il ouvre un modèle de document Word préexistant (template avec en-tête ministériel).
4. Il remplit manuellement chacun des champs du tableau (Cadre, Objet, Participants, Descriptif, Prochaine Action) en reformulant le contenu de l'email dans un style administratif formel.
5. Il enregistre le document et le transmet à sa hiérarchie pour information ou instruction.

Ce processus, bien que fonctionnel, présente un coût en temps non négligeable, en particulier lorsque le volume de correspondance est élevé ou lorsque plusieurs notes similaires doivent être produites dans un laps de temps réduit.

2.3 Limites du processus actuel

L'analyse de ce processus a permis d'identifier les limites suivantes :

- **Temps de traitement** : la rédaction manuelle d'une note complète prend en moyenne plusieurs minutes par email, ce qui devient significatif à l'échelle d'une direction traitant un volume important de correspondance.
- **Absence d'identification automatique du type de document** : l'agent doit ouvrir chaque pièce jointe pour en déterminer la nature avant de renseigner le champ « Cadre ».
- **Hétérogénéité rédactionnelle** : le style et le niveau de détail des notes varient d'un agent à l'autre, ce qui nuit à l'uniformité des documents transmis à la hiérarchie.
- **Absence de traçabilité automatisée** : aucun historique centralisé ne permet de retrouver rapidement quelles notes ont été produites à partir de quels emails.

2.4 Analyse des besoins fonctionnels

À partir de ce constat, les besoins fonctionnels suivants ont été formalisés :

ID	Besoin	Description
BF-01	Connexion à la messagerie Gmail	Le système doit pouvoir se connecter de manière sécurisée à la boîte Gmail de l'utilisateur et lister les emails récents.
BF-02	Récupération des pièces jointes	Le système doit extraire et sauvegarder les pièces jointes (PDF, images) associées à un email.
BF-03	Classification du document joint	Le système doit déterminer automatiquement le type de la pièce jointe parmi : Facture, Courrier officiel, Formulaire, Note interne.
BF-04	Extraction d'informations	Le système doit extraire de l'email l'objet, les participants, un descriptif et l'action à entreprendre.
BF-05	Reformulation administrative	Le contenu extrait doit être reformulé dans un style administratif formel avant intégration dans la note.
BF-06	Génération du document Word	Le système doit produire un fichier .docx respectant la charte graphique officielle (logos, tableau, mise en page).
BF-07	Prévisualisation avant téléchargement	L'utilisateur doit pouvoir visualiser le contenu généré avant de télécharger le document final.
BF-08	Téléchargement du document	L'utilisateur doit pouvoir télécharger la note générée au format .docx.

Table 2.1: Besoins fonctionnels identifiés

2.5 Analyse des besoins non fonctionnels

Au-delà des fonctionnalités, plusieurs exigences de qualité ont été prises en compte dans la conception du système :

- **Simplicité d'utilisation** : l'interface doit être accessible à un agent non technique, sans formation préalable.
- **Fidélité de la charte graphique** : le document généré doit être visuellement identique aux notes DSI existantes (logos, couleurs, polices, structure du tableau).
- **Contrôle humain** : le système doit rester un outil d'aide à la rédaction, l'agent devant pouvoir vérifier le contenu avant validation – aucune note ne doit être envoyée automatiquement sans supervision.
- **Confidentialité** : les données extraites des emails et les pièces jointes ne doivent pas transiter par des services tiers non maîtrisés ; le modèle de langage utilisé pour la reformulation est exécuté localement (Ollama).

- **Réutilisabilité** : l'architecture doit permettre d'ajouter facilement de nouveaux types de documents ou de nouveaux champs à la note sans réécrire l'ensemble du système.

2.6 Choix technologiques envisagés

Face à ces besoins, plusieurs choix technologiques ont été discutés et arbitrés :

- Pour la **classification de documents**, l'utilisation d'un réseau de neurones convolutif (CNN) a été retenue plutôt qu'une approche par règles ou mots-clés, celle-ci étant plus robuste face à la diversité visuelle des documents scannés.
- Pour l'**extraction et la reformulation du texte**, le recours à un modèle de langage (LLM) exécuté localement via **Ollama** a été privilégié à une solution basée sur une API cloud (type OpenAI), pour des raisons de confidentialité des données administratives.
- Pour l'**orchestration des différentes étapes de traitement** (classification, extraction, rédaction, génération), le framework **CrewAI** a été choisi car il permet de structurer le traitement sous la forme d'agents spécialisés collaborant séquentiellement, ce qui facilite la maintenance et l'évolution du système.
- Pour l'**interface utilisateur**, **Streamlit** a été retenu pour sa rapidité de mise en œuvre et son adéquation avec un prototype/outil interne à destination d'un nombre limité d'utilisateurs.

2.7 Conclusion

Ce chapitre a permis de formaliser les besoins du système à développer, à partir de l'analyse du processus manuel existant. Les besoins fonctionnels et non fonctionnels identifiés ici constituent le cahier des charges qui a guidé les choix de conception présentés dans le chapitre suivant.

Chapter 3

Conception de la solution

3.1 Introduction

Ce chapitre présente l'architecture globale de la solution développée, les choix de conception retenus pour chacun de ses modules, ainsi que les flux de données entre les différents composants. L'objectif est de donner une vision claire du fonctionnement interne du système avant d'en détailler l'implémentation dans le chapitre suivant.

3.2 Architecture globale

La solution repose sur une architecture modulaire composée de cinq grands blocs fonctionnels, s'exécutant séquentiellement :

1. un **module de connexion Gmail**, chargé de récupérer les emails et leurs pièces jointes ;
2. un **module de classification** basé sur un CNN, chargé de déterminer le type du document joint ;
3. un **système multi-agents** (CrewAI), chargé d'extraire et de reformuler les informations pertinentes ;
4. un **module de génération de document**, chargé de produire le fichier .docx final ;
5. une **interface utilisateur** (Streamlit), qui orchestre l'ensemble et permet l'interaction avec l'agent DSI.

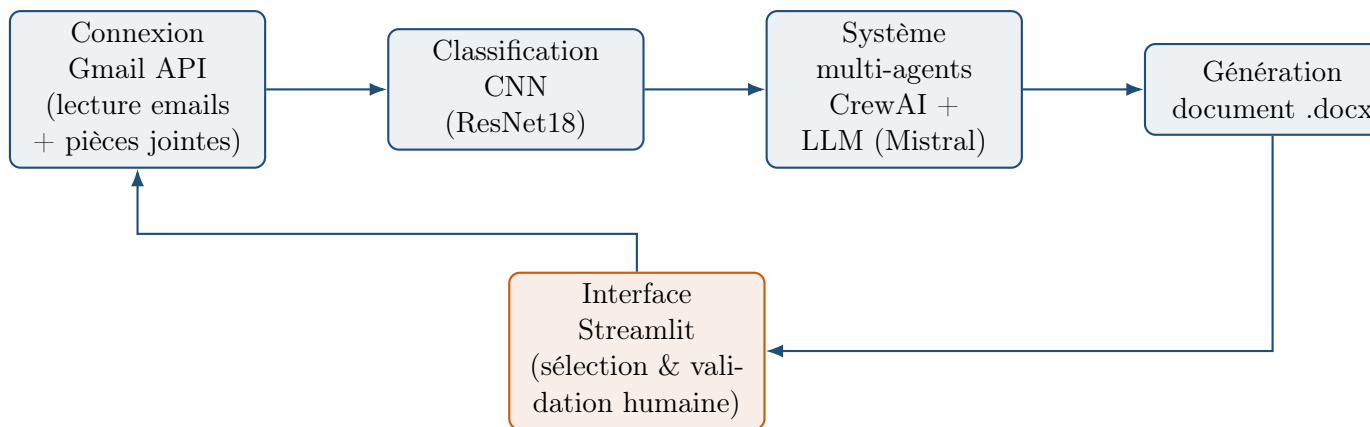


Figure 3.1: Architecture globale de la solution

L'interface Streamlit joue un rôle central : elle déclenche la récupération des emails, permet à l'utilisateur de sélectionner celui à traiter, lance le pipeline de traitement (classification → agents → génération), puis affiche le résultat pour validation avant téléchargement. Ce point de contrôle humain, positionné avant le téléchargement final, répond directement à l'exigence de supervision identifiée dans le chapitre précédent.

3.3 Conception du module de classification (CNN)

3.3.1 Choix de l'architecture

Pour la classification des documents joints, l'architecture retenue est **ResNet18**, un réseau de neurones convolutif profond popularisé par les travaux de He et al., reconnu pour son bon compromis entre performance et légèreté computationnelle. Plutôt que d'entraîner ce réseau depuis zéro, une stratégie de **transfert d'apprentissage (transfer learning)** a été adoptée : les poids pré-entraînés sur ImageNet sont conservés et gelés pour l'ensemble des couches convolutives, seule la tête de classification (couches entièrement connectées) étant réentraînée sur le jeu de données spécifique aux documents administratifs.

Cette approche présente deux avantages majeurs dans le contexte du stage : un temps d'entraînement fortement réduit, et une bonne capacité de généralisation malgré un jeu de données d'entraînement de taille limitée.

3.3.2 Classes de documents

Quatre classes de documents ont été définies en concertation avec la DSI, correspondant aux types de pièces jointes les plus fréquemment reçues :

- **Facture** : documents comptables/financiers ;
- **Courrier officiel** : correspondance administrative formelle ;
- **Formulaire** : documents à champs à remplir ;
- **Note interne** : notes de service ou communications internes.

3.3.3 Stratégie de constitution du jeu de données

En l'absence d'un corpus existant de documents réels étiquetés (pour des raisons de confidentialité des données administratives), la décision a été prise de **générer un jeu de données synthétique** reproduisant la structure visuelle typique de chaque classe de document (titres, tableaux, lignes de texte, zones de signature), avec ajout de bruit pour simuler les variations de qualité de numérisation. Cette approche est détaillée dans le chapitre suivant.

3.4 Conception du système multi-agents (CrewAI)

3.4.1 Principe général

Le traitement du texte de l'email (extraction d'informations et reformulation) est confié à un système multi-agents construit avec le framework **CrewAI**. Ce choix architectural permet de décomposer une tâche complexe – transformer un email brut en note administrative structurée – en plusieurs sous-tâches spécialisées, chacune confiée à un agent dédié, s'exécutant séquentiellement (`Process.sequential`) et se transmettant leurs résultats via un mécanisme de contexte.

3.4.2 Les quatre agents

Quatre agents ont été définis, chacun avec un rôle, un objectif et un contexte (back-story) précis :

Agent	Rôle
Classificateur	Détermine le type du document joint en invoquant l'outil de classification CNN (<code>classify_document_tool</code>).
Extracteur	Analyse le texte de l'email et extrait les informations clés : objet, participants, contenu, action à mener.
Rédacteur	Reformule chaque champ extrait dans un style administratif formel, en français.
Générateur	Consolide le contenu final structuré (JSON) transmis ensuite au module de génération de document.

Table 3.1: Rôles des quatre agents du système CrewAI

3.4.3 Choix du modèle de langage

Chaque agent s'appuie sur un modèle de langage **Mistral**, exécuté localement via **Ollama**. Ce choix répond à l'exigence de confidentialité identifiée dans l'analyse des besoins : les données des emails, potentiellement sensibles, ne transitent par aucune API externe et restent traitées sur l'infrastructure locale.

3.4.4 Flux d'exécution des tâches

Les quatre tâches (**Task**) associées aux agents s'enchaînent de façon séquentielle, chaque tâche recevant en contexte le résultat de la précédente :

Classification → Extraction → Rédaction → Génération

Un point de conception important, identifié lors des tests, mérite d'être souligné : bien que le résultat de la classification transite formellement dans le pipeline CrewAI, le **résultat affiché à l'utilisateur dans l'interface** est recalculé directement en appelant la fonction Python `classify_document_file()`, indépendamment de la sortie du LLM. Ce choix de conception, permet d'éviter que le modèle de langage ne reformule ou n'altère le résultat brut et fiable du CNN.

3.5 Conception du module de génération de document

Le module de génération s'appuie sur la bibliothèque **python-docx** pour produire un fichier Word respectant strictement la charte graphique des notes DSI existantes : en-tête avec logo du Ministère, pied de page avec logo « Génération Green » et coordon-

nées de la DSI, tableau structuré à six lignes (Cadre, Objet, Participants, Descriptif, Prochaine Action, Instructions Sollicitées), avec mise en forme des bordures, couleurs et hauteurs de lignes gérée au niveau XML (OOXML) pour un rendu fidèle au modèle papier original.

3.6 Conception de l'interface utilisateur

L'interface a été conçue avec **Streamlit** selon un parcours utilisateur en quatre étapes linéaires, correspondant chacune à une section de la page :

1. **Chargement des emails** : bouton de connexion à Gmail et affichage de la liste des emails récents.
2. **Sélection de l'email** : liste des emails avec objet, expéditeur, date et pièces jointes ; bouton « Sélectionner ».
3. **Génération de la note** : aperçu du contenu de l'email sélectionné, bouton « Générer la note DSI » déclenchant le pipeline complet.
4. **Aperçu et téléchargement** : affichage du type de document détecté et des champs générés, avec bouton de téléchargement du fichier .docx final.

Ce parcours garantit qu'aucune note n'est générée sans action explicite de l'utilisateur, et qu'aucun téléchargement n'a lieu sans que celui-ci ait pu vérifier visuellement le contenu produit – répondant ainsi directement au besoin non fonctionnel de contrôle humain.

3.7 Conclusion

Ce chapitre a détaillé l'architecture et les choix de conception retenus pour chacun des modules de la solution. Le chapitre suivant présente la réalisation concrète de ces différents modules, avec les extraits de code et résultats obtenus.

Chapter 4

Réalisation

4.1 Introduction

Ce chapitre présente en détail la réalisation technique de chacun des modules décrits au chapitre précédent : l'entraînement du modèle de classification CNN, la mise en place du système multi-agents CrewAI, l'intégration avec l'API Gmail, le module de génération de documents Word, ainsi que l'interface utilisateur Streamlit. Des captures d'écran du code source illustrent les choix d'implémentation les plus significatifs.

4.2 Environnement de développement

Le développement a été réalisé sous **Visual Studio Code**, en langage **Python**. La figure 4.2 présente l'arborescence du projet telle qu'organisée dans l'environnement de développement.

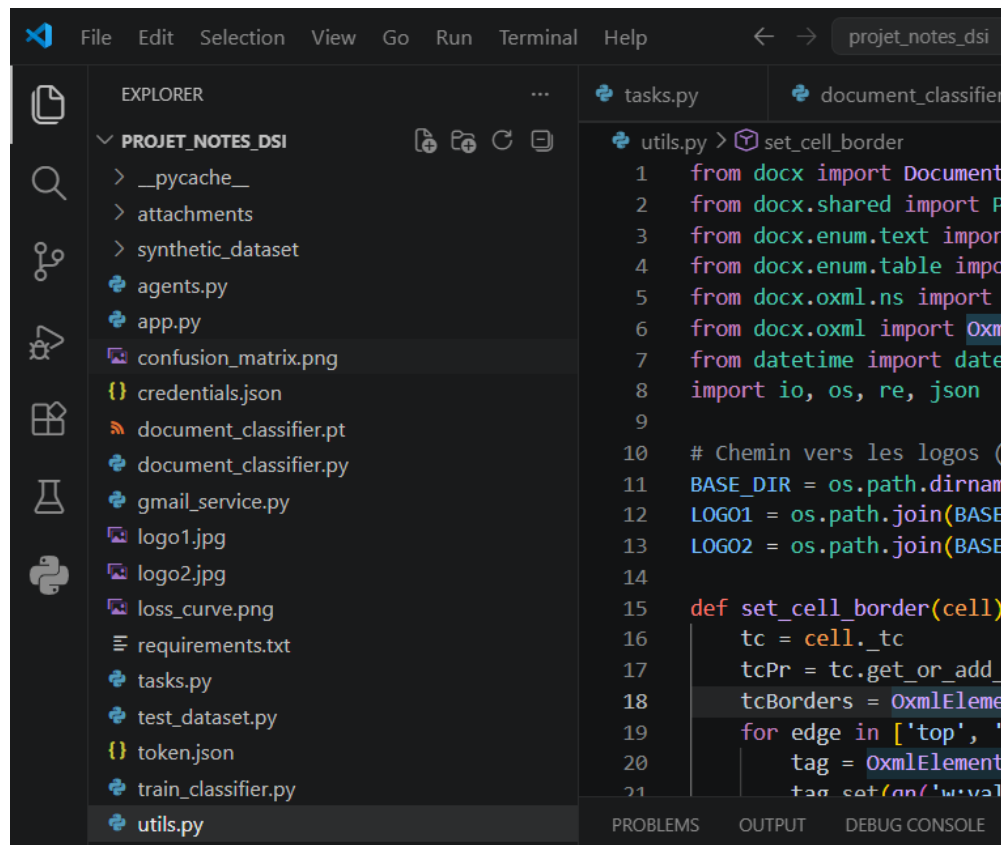
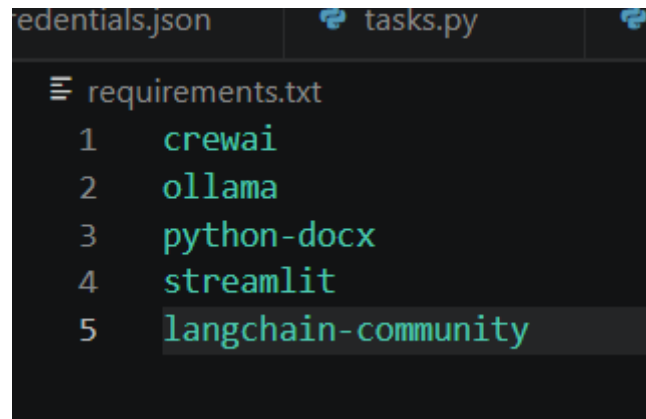


Figure 4.1: Arborescence du projet dans Visual Studio Code

Le projet est structuré en modules indépendants et clairement séparés :

- `app.py` : point d'entrée de l'interface Streamlit ;
- `agents.py` : définition des quatre agents CrewAI ;
- `tasks.py` : définition des tâches associées à chaque agent ;
- `gmail_service.py` : intégration avec l'API Gmail ;
- `document_classifier.py` : chargement du modèle CNN et outil de classification;
- `train_classifier.py` : script d'entraînement du CNN;
- `utils.py` : génération du document Word et fonctions utilitaires ;
- `attachments/` : dossier de stockage des pièces jointes téléchargées ;
- `synthetic_dataset/` : jeu de données synthétique généré pour l'entraînement.

Les principales bibliothèques Python utilisées sont listées dans le fichier `requirements.txt`, présenté en figure 4.2.

A screenshot of a code editor window. The top bar shows two tabs: 'credentials.json' and 'tasks.py'. The main editor area is titled 'requirements.txt' and contains a list of five dependencies, each on a new line and numbered from 1 to 5. The dependencies are: 1 crewai, 2 ollama, 3 python-docx, 4 streamlit, and 5 langchain-community. The text is in a light green color on a dark background.

```
1 crewai
2 ollama
3 python-docx
4 streamlit
5 langchain-community
```

Figure 4.2: Dépendances principales du projet (requirements.txt)

4.3 Entraînement du modèle de classification (CNN)

4.3.1 Génération du jeu de données synthétique

Comme évoqué au chapitre précédent, un jeu de données synthétique a été généré localement, sans téléchargement externe, afin de constituer un corpus d'entraînement pour les quatre classes de documents (Facture, Courrier officiel, Formulaire, Note interne). Chaque image est générée par programme à l'aide de la bibliothèque `Pillow` : un fond blanc de format A4, sur lequel sont dessinés des éléments typiques de chaque type de document (titres, tableaux, lignes de texte simulées, zones de signature), puis un bruit gaussien léger est appliqué pour simuler les artefacts de numérisation. La fonction `generate_image()` illustre ce principe pour la classe « Facture », en dessinant un titre, un numéro de facture aléatoire, un tableau et un montant TTC, avant l'ajout du bruit.

```

31
32
33 def generate_image(doc_type, idx):
34     img = Image.new("RGB", (600, 800), color="white")
35     draw = ImageDraw.Draw(img)
36     try:
37         font_title = ImageFont.truetype("arial.ttf", 26)
38         font_text = ImageFont.truetype("arial.ttf", 16)
39     except Exception:
40         font_title = ImageFont.load_default()
41         font_text = ImageFont.load_default()
42
43     if doc_type == "Facture":
44         draw.text((30, 30), "FACTURE", font=font_title, fill="black")
45         draw.text((30, 80), f"78 FAC (1800xide)", font=font_text, fill="black")
46         draw.rectangle([30, 150, 570, 400], outline="black", width=2)
47         for y in range(170, 380, 40):
48             draw.line([(30, y), (570, y)], fill="gray", width=1)
49         draw.text((30, 420), f"Total TTC : {random.randint(100,5000)} MAD", font=font_text, fill="black")
50
51     elif doc_type == "Courrier officiel":
52         draw.text((30, 30), "OBJET : Correspondance officielle", font=font_title, fill="black")
53         draw.text((30, 180), "Madame, Monsieur,", font=font_text, fill="black")
54         for y in range(140, 500, 30):
55             draw.line([(30, y), (random.randint(400, 570), y)], fill="black", width=1)
56         draw.text((30, 550), "Veuillez agréer, Madame, Monsieur,", font=font_text, fill="black")
57         draw.text((30, 580), "l'expression de mes salutations distinguées.", font=font_text, fill="black")
58
59     elif doc_type == "Formulaire":
60         draw.text((30, 30), "FORMULAIRE", font=font_title, fill="black")
61         for i, label in enumerate(["Nom :", "Prenom :", "Date :", "Adresse :", "Signature :"]):
62             y = 100 + i * 50
63             draw.text((30, y), label, font=font_text, fill="black")
64             draw.rectangle([180, y - 5, 570, y + 30], outline="black", width=1)
65
66     elif doc_type == "Note interne":
67         draw.text((30, 30), "NOTE INTERNE", font=font_title, fill="black")

```

Figure 4.3: Génération d'une image synthétique de type "Facture"

Au total, **120 images par classe** ont été générées, soit **480 images** réparties sur les quatre catégories, puis divisées en un ensemble d'entraînement (80 %) et un ensemble de validation (20 %).

4.3.2 Architecture du modèle

Le modèle repose sur **ResNet18** pré-entraîné sur ImageNet, dont les couches convolutives sont gelées. Seule la tête de classification est réentraînée, remplacée par un petit réseau entièrement connecté avec deux couches de *dropout* (taux 0.4) afin de limiter le surapprentissage compte tenu de la taille réduite du jeu de données. La fonction `build_model()` construit ce réseau en gelant les paramètres du backbone puis en remplaçant la couche finale (`model.fc`) par une séquence Dropout → Linéaire → ReLU → Dropout → Linéaire.

```

def build_model(num_classes, dropout=DROPOUT):
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    for param in model.parameters():
        param.requires_grad = False
    in_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(p=dropout),
        nn.Linear(in_features, 128),
        nn.ReLU(),
        nn.Dropout(p=dropout),
        nn.Linear(128, num_classes)
    )
    return model.to(DEVICE)

```

Figure 4.4: Construction du modèle ResNet18 adapté

Les hyperparamètres d'entraînement retenus sont résumés dans le tableau 4.1.

Hyperparamètre	Valeur
Taille d'image en entrée	224×224
Taille de batch	16
Nombre d'époques	6
Taux de dropout	0.4
Optimiseur	Adam
Taux d'apprentissage	1×10^{-4}
Fonction de perte	Entropie croisée (CrossEntropyLoss)

Table 4.1: Hyperparamètres utilisés pour l'entraînement du modèle CNN

4.3.3 Boucle d'entraînement

L'entraînement suit une boucle classique en PyTorch : passage avant (*forward*), calcul de la perte, rétropropagation, mise à jour des poids. Seuls les paramètres de la tête de classification sont optimisés, les poids du backbone ResNet18 restant gelés.

```

124
125 def train_one_epoch(model, loader, optimizer, criterion):
126     model.train()
127     total_loss = 0
128     for imgs, labels in loader:
129         imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
130         optimizer.zero_grad()
131         outputs = model(imgs)
132         loss = criterion(outputs, labels)
133         loss.backward()
134         optimizer.step()
135         total_loss += loss.item() * imgs.size(0)
136     return total_loss / len(loader.dataset)
137

```

Figure 4.5: Boucle d'entraînement d'une époque

4.3.4 Résultats de l'entraînement

La figure 4.3.4 présente l'évolution de la perte d'entraînement et de validation au fil des six époques. On observe une décroissance régulière et conjointe des deux courbes, sans signe de surapprentissage marqué, ce qui valide le choix du taux de dropout et de la stratégie de gel des couches convolutives.

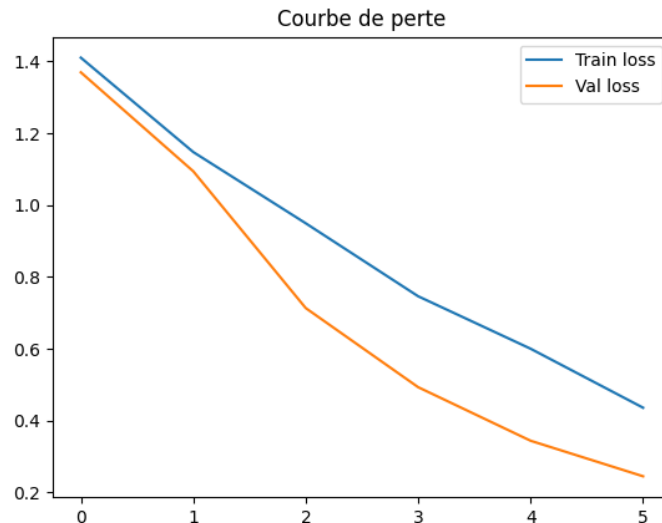


Figure 4.6: Courbe de perte (entraînement et validation) sur 6 époques

La matrice de confusion obtenue sur l'ensemble de validation (figure 4.3.4) illustre d'excellentes performances de classification, avec une très faible confusion entre les quatre classes.

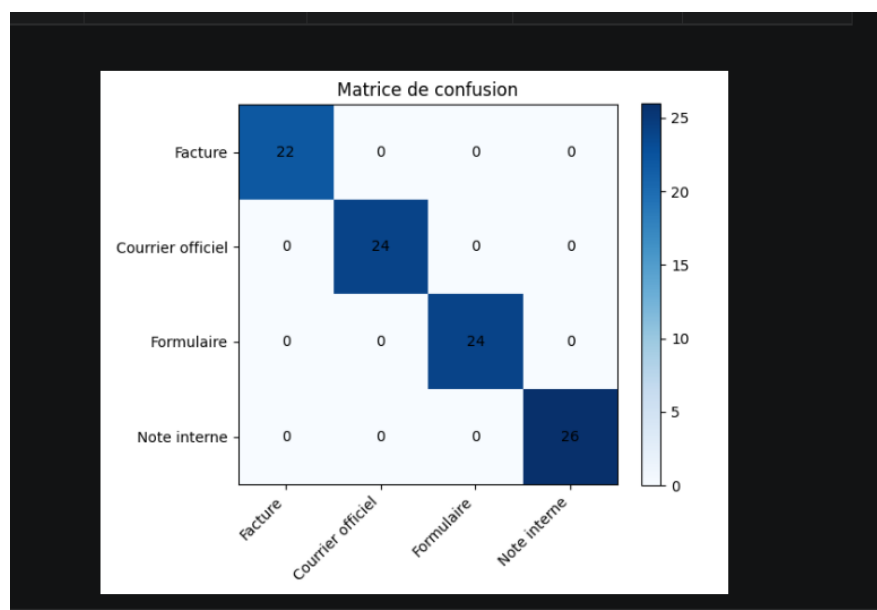


Figure 4.7: Matrice de confusion du modèle CNN sur l'ensemble de validation

Le modèle atteint une exactitude (*accuracy*) globale supérieure à 95 % sur l'ensemble de validation, avec un taux de classification correcte proche de 100 % pour les classes « Courrier officiel », « Formulaire » et « Note interne », et une seule confusion mineure observée sur la classe « Facture ». Ces résultats confirment la pertinence de l'approche par transfert d'apprentissage, même avec un jeu de données synthétique de taille mod-

este.

4.3.5 Exposition du modèle comme outil CrewAI

Une fois entraîné, le modèle est sauvegardé au format `.pt` et chargé de manière paresseuse (*lazy loading*) lors de la première utilisation, afin d'éviter de recharger les poids à chaque appel. Le modèle est ensuite exposé sous la forme d'un outil CrewAI (@tool), directement utilisable par l'agent « Classificateur ». La fonction `classify_document_file()` charge l'image (ou convertit la première page d'un PDF en image), applique les transformations nécessaires, puis retourne l'étiquette prédite ainsi que le niveau de confiance associé.

```
72 def classify_document_file(file_path: str) -> dict:
73     """
74     Classifie un fichier (PDF ou image) et retourne :
75     {"label": "Facture", "confidence": 0.87}
76     """
77     model, class_names = _load_model()
78
79     ext = os.path.splitext(file_path)[1].lower()
80     if ext == ".pdf":
81         img = _pdf_to_image(file_path)
82     else:
83         img = Image.open(file_path).convert("RGB")
84
85     tensor = _transform(img).unsqueeze(0).to(DEVICE)
86
87     with torch.no_grad():
88         outputs = model(tensor)
89         probs = torch.softmax(outputs, dim=1)[0]
90         pred_idx = int(torch.argmax(probs).item())
91         confidence = float(probs[pred_idx].item())
92
93     return {
94         "label": class_names[pred_idx],
95         "confidence": round(confidence, 3)
96     }
97
```

Figure 4.8: Chargement paresseux du modèle et fonction de classification

Un point notable de l'implémentation est la prise en charge des pièces jointes au format **PDF** : celles-ci sont automatiquement converties en image (première page) via la bibliothèque `pdf2image`, avant d'être soumises au modèle – la très grande majorité des pièces jointes reçues par email étant au format PDF plutôt qu'image.

4.4 Système multi-agents avec CrewAI

4.4.1 Définition des agents

Les quatre agents présentés au chapitre précédent sont définis en Python à l'aide du framework CrewAI. Chaque agent est configuré avec un rôle, un objectif, un contexte narratif (*backstory*) et, pour l'agent classificateur, un outil dédié : l'agent Classificateur reçoit ainsi le rôle de spécialiste de la reconnaissance de documents administratifs, avec pour consigne stricte de n'utiliser que l'outil de classification CNN et de ne jamais deviner un type au jugé.

```
classificateur = Agent(  
    role="Classificateur de documents",  
    goal=(  
        "Déterminer le type du document joint à l'email (Facture, courrier officiel, "  
        "Formulaire, Note interne) en utilisant l'outil de classification basé sur un "  
        "CNN entraîné. Si aucune pièce jointe n'est fournie, répondre 'Aucun document'.",  
    ),  
    backstory=(  
        "Tu es un spécialiste de la reconnaissance de documents administratifs. "  
        "Tu utilises exclusivement l'outil de classification fourni pour déterminer "  
        "le type de document, tu n'inventes jamais un type au jugé.",  
    ),  
    tools=[classify_document_tool],  
    llm="ollama/mistral",  
    verbose=True  
)
```

Figure 4.9: Définition de l'agent Classificateur

Les trois autres agents (Extracteur, Rédacteur, Générateur) suivent une structure similaire, sans outil dédié, et s'appuient uniquement sur le modèle de langage ollama/mistral pour accomplir leur tâche respective.

4.4.2 Définition des tâches

Chaque agent se voit associer une tâche (*Task*) précisant la description du travail attendu, le format de sortie attendu (*expected_output*) et, le cas échéant, le contexte issu des tâches précédentes. La tâche d'extraction, particulièrement structurante, impose une sortie strictement au format JSON avec cinq champs (cadre, objet, participants, descriptif, prochaine_action), accompagnée de règles strictes de rédaction (résumé détaillé en 3-4 phrases, action concrète à mener, participants issus uniquement de l'email).

```

2   tache_extraction = Task(
3       description="""
4       Tu es un extracteur d'informations. Analyse cet email et réponds UNIQUEMENT avec du JSON.
5
6       EMAIL À ANALYSER :
7       {email_text}
8
9       Tu disposes aussi du type de document détecté par le classificateur (voir contexte).
10      Utilise cette information pour renseigner le champ "cadre" si c'est pertinent
11      (par exemple si le document est une "Facture", le cadre peut être "financier").
12
13      RÈGLES ABSOLUES :
14      1. Réponds SEULEMENT avec le JSON, rien d'autre
15      2. Le "descriptif" = résumé DÉTAILLÉ du contenu (3-4 phrases) :
16         mentionne les informations clés comme le RIB, les documents joints,
17         les instructions données dans l'email
18      3. La "prochaine_action" = action concrète à faire suite à cet email
19         (ne jamais laisser vide)
20      4. Les "participants" = noms/emails présents dans l'email uniquement
21      5. NE JAMAIS copier les instructions dans le JSON
22
23      JSON à retourner :
24      {
25          "cadre": "...",
26          "objet": "...",
27          "participants": "...",
28          "descriptif": "...",
29          "prochaine_action": "..."
30      }
31      """
32      ,
33      expected_output="JSON valide uniquement, 5 champs string.",
34      agent=extracteur,
35      context=[tache_classification]
36  )

```

Figure 4.10: Extrait de la tâche d'extraction d'informations

Les quatre tâches sont ensuite regroupées au sein d'un objet **Crew**, exécuté selon un processus séquentiel (`Process.sequential`), garantissant que chaque agent dispose bien du résultat de l'agent précédent avant de démarrer son propre traitement.

4.4.3 Choix de conception : classification affichée hors LLM

Un point de conception important a été introduit suite à des tests préliminaires : il a été constaté que le modèle de langage Mistral, sollicité via l'agent Rédacteur, avait tendance à **reformuler ou halluciner** le résultat de la classification transmis par l'agent Classificateur, au lieu de le restituer tel quel. Afin de garantir la fiabilité de l'information affichée à l'utilisateur – un des besoins non fonctionnels identifiés au chapitre 2 –, il a été décidé d'appeler **directement en Python** la fonction `classify_document_file()` pour l'affichage dans l'interface, en parallèle de l'exécution du pipeline CrewAI.

4.5 Intégration avec l'API Gmail

4.5.1 Authentification OAuth 2.0

L'accès à la boîte Gmail de l'utilisateur repose sur le protocole **OAuth 2.0**, via la bibliothèque officielle `google-auth-oauthlib`. Lors de la première exécution, l'utilisateur est redirigé vers une page d'autorisation Google ; le jeton d'accès obtenu est ensuite stocké localement et automatiquement rafraîchi lors des exécutions suivantes, via la fonction `get_gmail_service()`.

```
def get_gmail_service():
    creds = None
    token_path = os.path.join(BASE_DIR, 'token.json')
    creds_path = os.path.join(BASE_DIR, 'credentials.json')

    if os.path.exists(token_path):
        creds = Credentials.from_authorized_user_file(token_path, SCOPES)

    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            flow = InstalledAppFlow.from_client_secrets_file(creds_path, SCOPES)
            creds = flow.run_local_server(port=0)
            with open(token_path, 'w') as token:
                token.write(creds.to_json())

    return build('gmail', 'v1', credentials=creds)
```

Figure 4.11: Authentification et création du service Gmail

L'accès demandé est volontairement limité au périmètre `gmail.readonly`, restreignant l'application à la seule lecture des emails, sans possibilité d'envoi ni de modification – un choix de sécurité minimisant la surface d'exposition de l'application.

4.5.2 Extraction du corps de l'email et des pièces jointes

Les emails Gmail étant structurés sous forme de parties MIME potentiellement imbriquées (texte brut, HTML, pièces jointes), une fonction récursive d'aplatissement (`_walk_parts`) a été implémentée afin de parcourir l'ensemble de l'arborescence, quelle que soit sa profondeur d'imbrication.

```
59
60 def _walk_parts(parts):
61     """Aplati récursivement l'arborescence de parts d'un email (les pièces
62     jointes peuvent être nichées dans des sous-parts multipart/mixed)."""
63     flat = []
64     for part in parts:
65         if 'parts' in part:
66             flat.extend(_walk_parts(part['parts']))
67         else:
68             flat.append(part)
69     return flat
```

Figure 4.12: Aplatissement récursif des parties MIME d'un email

Seules les pièces jointes de type PDF ou image (PNG, JPEG) sont conservées, conformément au périmètre du modèle de classification ; les autres types de fichiers sont ignorés.

4.6 Génération du document Word

4.6.1 Reproduction fidèle de la charte graphique

Le module `utils.py` s'appuie sur la bibliothèque **python-docx** pour générer le document final. La fidélité visuelle avec les notes DSI existantes a nécessité une manipulation fine du format OOXML sous-jacent, notamment pour la gestion des bordures de cellules (non supportée nativement par l'API haut niveau de `python-docx`), gérée par la fonction `set_cell_border()`.

```

15 def set_cell_border(cell):
16     tc = cell._tc
17     tcPr = tc.get_or_add_tcPr()
18     tcBorders = XmlElement('w:tcBorders')
19     for edge in ['top', 'left', 'bottom', 'right']:
20         tag = XmlElement(f'w:{edge}')
21         tag.set(qn('w:val'), 'single')
22         tag.set(qn('w:sz'), '4')
23         tag.set(qn('w:space'), '0')
24         tag.set(qn('w:color'), '000000')
25         tcBorders.append(tag)
26     tcPr.append(tcBorders)
27

```

Figure 4.13: Ajout manuel des bordures de cellule en XML (OOXML)

Le document généré reprend fidèlement les éléments suivants :

- l'en-tête avec le logo officiel du Ministère (armoiries du Royaume du Maroc, texte bilingue français/arabe/amazigh) ;
- le pied de page avec le logo « Génération Green 2020–2030 » et les coordonnées de la DSI ;
- un tableau à six lignes, avec les libellés de champs en bleu et gras, respectant les hauteurs de ligne adaptées au contenu (lignes plus hautes pour les champs « Descriptif » et « Prochaine Action ») ;
- la mention « DSI » et la date du jour, générée dynamiquement.

4.6.2 Structuration du tableau de la note

Le tableau final regroupe les six champs de la note DSI, dont cinq sont issus du traitement par les agents IA, le sixième (« Instructions Sollicitées ») étant une mention fixe standard.

```
100 rows_data = [  
101     ("Cadre", data.get("cadre", ""), False),  
102     ("Objet", data.get("objet", ""), False),  
103     ("Participants", data.get("participants", ""), False),  
104     ("Descriptif", data.get("descriptif", ""), True),  
105     ("Prochaine Action", data.get("prochaine_action", ""), True),  
106     ("Instruction Sollicités", "Pour votre information et éventuelles instructions.", False),  
107 ]  
108
```

Figure 4.14: Construction des lignes du tableau de la note

4.6.3 Robustesse du parsing JSON

La sortie du dernier agent (Générateur) étant un texte potentiellement entouré de commentaires ou de balises Markdown malgré les consignes strictes données, une fonction de récupération (`parse_json_from_text`) a été implémentée afin d'extraire de manière robuste l'objet JSON recherché, avec repli sur des valeurs par défaut (« Non déterminé ») en cas d'échec de parsing – garantissant qu'aucune erreur ne bloque la génération du document final.

4.7 Conclusion

Ce chapitre a détaillé la réalisation technique des principaux modules du système : classification par CNN, orchestration multi-agents, intégration Gmail et génération de documents Word. Le chapitre suivant présente l'interface utilisateur finale ainsi que les tests et résultats obtenus sur l'application complète.

Chapter 5

Tests et résultats

5.1 Introduction

Ce chapitre présente le fonctionnement de l'application finale à travers un scénario d'utilisation complet, de la connexion à Gmail jusqu'au téléchargement de la note DSI générée. Les captures d'écran successives illustrent chaque étape du parcours utilisateur défini au chapitre 3, ainsi que le document Word final obtenu.

5.2 Interface d'accueil

Au lancement de l'application (commande `streamlit run app.py`), l'utilisateur accède à une interface web minimaliste présentant le titre de l'application et le bouton permettant de charger les emails récents depuis Gmail.



Figure 5.1: Page d'accueil de l'application Streamlit

5.3 Chargement et sélection d'un email

Après un clic sur le bouton « Charger les emails », l'application se connecte à la boîte Gmail via l'API et affiche la liste des emails récents, avec pour chacun l'objet, l'expéditeur, la date et, le cas échéant, la liste des pièces jointes détectées.

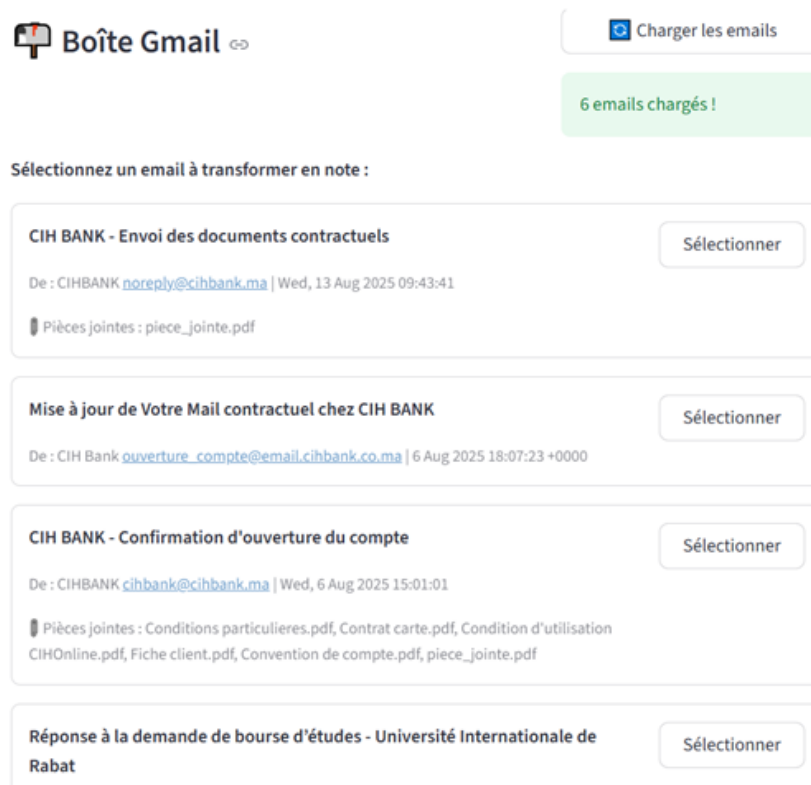


Figure 5.2: Liste des emails chargés depuis Gmail, avec pièces jointes détectées

On observe sur cette capture que l'application identifie correctement plusieurs emails avec leurs pièces jointes respectives (par exemple, l'email « CIH BANK – Confirmation d'ouverture du compte » comporte cinq pièces jointes PDF distinctes). L'utilisateur sélectionne ensuite l'email à transformer en note en cliquant sur le bouton « Sélectionner » correspondant.

Une fois un email sélectionné, son contenu détaillé est affiché dans un panneau dédié, avec la pièce jointe détectée mise en évidence.



Figure 5.3: Aperçu de l'email sélectionné avant génération

5.4 Génération de la note

Le clic sur le bouton « Générer la note DSI » déclenche l'exécution du pipeline complet : classification du document joint par le CNN, puis extraction et reformulation des informations par le système multi-agents CrewAI. Un indicateur de chargement informe l'utilisateur que les agents sont en cours de traitement.

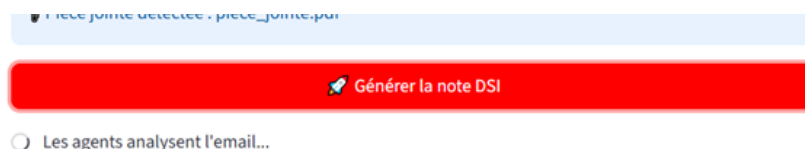


Figure 5.4: Indicateur de traitement pendant l'exécution du pipeline multi-agents

Ce traitement mobilise séquentiellement les quatre agents définis au chapitre précédent ; sa durée dépend principalement du temps de réponse du modèle de langage local (Mistral via Ollama), généralement de l'ordre de quelques dizaines de secondes selon la longueur de l'email traité.

5.5 Aperçu et validation du résultat

Une fois le traitement terminé, l'interface affiche un aperçu structuré de la note générée, comprenant :

- le type de document détecté par le modèle CNN, avec son niveau de confiance ;
- les cinq champs de la note (Cadre, Objet, Participants, Descriptif, Prochaine Action), reformulés en français administratif.

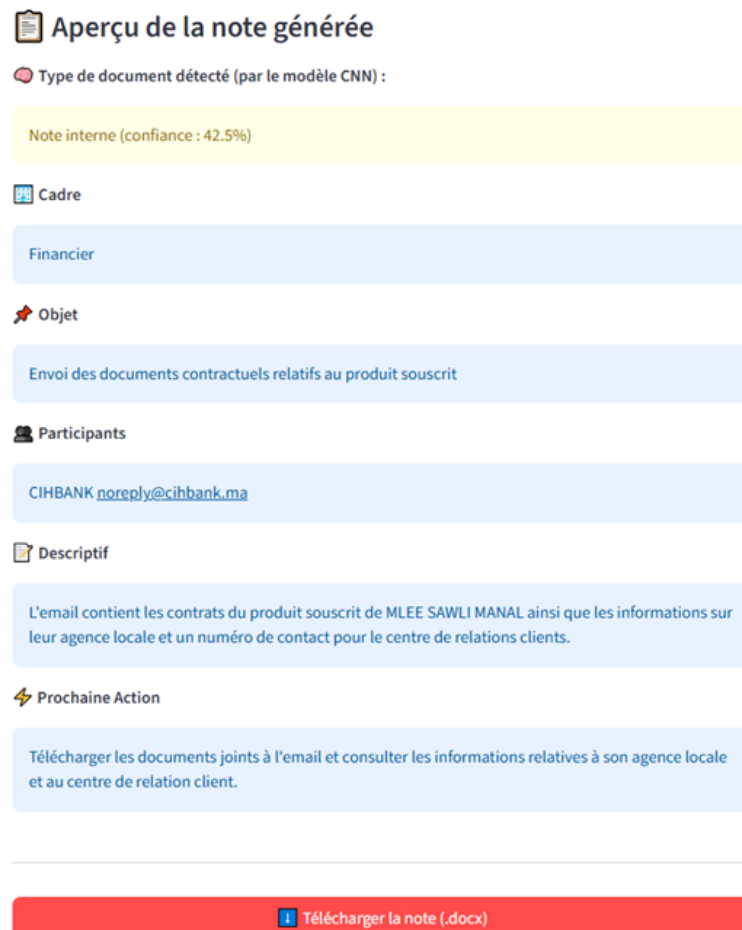


Figure 5.5: Aperçu de la note DSI générée avant téléchargement

Cette étape constitue le point de contrôle humain identifié comme besoin non fonctionnel prioritaire : l'utilisateur peut vérifier la pertinence du contenu généré – en particulier le type de document détecté par le CNN, ici « Note interne » avec une confiance de 42,5 % – avant de procéder au téléchargement. On notera que ce niveau de confiance relativement modéré, obtenu sur un document réel (par opposition aux données synthétiques d'entraînement), illustre l'écart de performance attendu entre données synthétiques et données réelles, discuté plus loin dans ce chapitre.

5.6 Téléchargement et document final

Le clic sur le bouton « Télécharger la note (.docx) » déclenche le téléchargement du fichier Word généré par le navigateur.

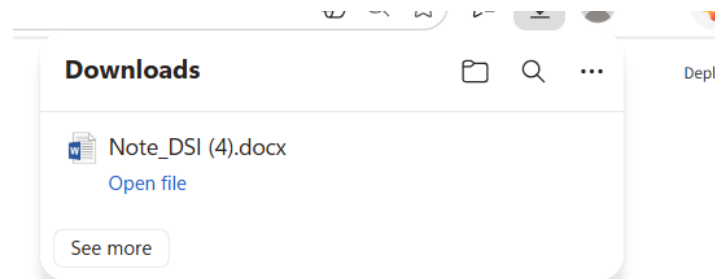


Figure 5.6: Téléchargement du fichier Note_DSI.docx

Le document Word obtenu, ouvert dans Microsoft Word, reproduit fidèlement la charte graphique officielle des notes DSI du Ministère : en-tête avec les armoiries du Royaume, tableau structuré avec libellés en bleu, et pied de page avec le logo « Génération Green » et les coordonnées de la Direction des Systèmes d'Information.

Note_DSI (4) [Mode de compatibilité] - Word (Échec de l'activation du produit)

blipostage Révision Affichage Dites-nous ce que vous voulez faire..

المملكة المغربية
ROYAUME DU MAROC

وزارة المصايد والصيد البحري والتنمية القروية والمياه والغابات
Ministère de l'Agriculture, de la Pêche Maritime, du Développement Rural et des Eaux et Forêts

DSI Rabat, le 11/07/2026

Note

Cadre	Financier
Objet	Envoi des documents contractuels relatifs au produit souscrit
Participants	CIHBANK <noreply@cihbank.ma>
Descriptif	L'email contient les contrats du produit souscrit de MLEE SAWLI MANAL ainsi que les informations sur leur agence locale et un numéro de contact pour le centre de relations clients.
Prochaine Action	Télécharger les documents joints à l'email et consulter les informations relatives à son agence locale et au centre de relation client.
Instruction Sollicités	Pour votre information et éventuelles instructions.


 DIRECTION DES SYSTÈMES D'INFORMATION
 Avenue Mad V-Quartier Administratif
 place Abdelkhalq Chelchewant Rabat - Chellah
 Tél.: (+212) 537 10 31 80/31 81 Fax : (+212) 537 76 13 64

مديرية نظم المعلومات
 ساحة عبد الله الشاذلي الرباط - شالة
 شارع محمد الخامس أبي العلاء
 (+212) 537 10 31 80/31 81 الهاتف : (+212) 537 76 13 64

Figure 5.7: Note DSI finale générée, ouverte dans Microsoft Word

5.7 Analyse des résultats

5.7.1 Performance du modèle de classification

Comme présenté au chapitre 4, le modèle CNN entraîné sur le jeu de données synthétique atteint une exactitude supérieure à 95 % en validation. Toutefois, le test sur un document réel (capture de la section précédente) montre un niveau de confiance plus modéré (42,5 %) pour la classe prédite. Cet écart s'explique par la différence de distribu-

tion entre les images synthétiques générées par programme (mise en page géométrique régulière) et les documents réels rencontrés en production (mise en page variable, polices différentes, filigranes, tampons, qualité de numérisation hétérogène). Cette observation constitue une piste d'amélioration identifiée en conclusion de ce rapport.

5.7.2 Pertinence du contenu généré par les agents

L'examen du contenu généré (figure de la section précédente) montre que le système parvient à produire un résumé cohérent et pertinent du contenu de l'email, avec une reformulation administrative appropriée. Le champ « Cadre » (« Financier ») est correctement déduit du contexte de l'email (communication d'un établissement bancaire), et la « Prochaine Action » proposée est cohérente avec le contenu de l'email source.

5.7.3 Fiabilité de l'architecture retenue

Le choix de séparer le résultat de classification affiché à l'utilisateur (appel direct de la fonction Python) de la sortie du pipeline LLM s'est révélé pertinent lors des tests : plusieurs exécutions ont confirmé que le modèle de langage, livré à lui-même, reformulait parfois le résultat de classification de façon incorrecte, alors que l'appel direct restitue toujours fidèlement la prédiction réelle du CNN.

5.7.4 Limites observées

Les tests réalisés ont permis d'identifier plusieurs limites du système dans son état actuel :

- le **temps de traitement** du pipeline complet reste perceptible (pouvant atteindre une trentaine de minutes par e-mail), en raison de l'exécution locale du modèle de langage sur un poste de travail standard, sans accélération GPU dédiée;
- l'**écart de performance** du modèle CNN entre données synthétiques et documents réels, mentionné ci-dessus ;
- l'application traite pour l'instant **une seule pièce jointe** par email (la première détectée), alors que certains emails en comportent plusieurs ;
- l'**absence de persistance** : les notes générées ne sont pas historisées dans une base de données, chaque session étant indépendante.

5.8 Conclusion

Ce chapitre a permis de démontrer, à travers un scénario d'utilisation complet illustré par des captures d'écran de l'application réelle, le bon fonctionnement du système développé. Les résultats obtenus, tant sur le plan de la classification automatique que de la génération de contenu textuel, valident l'approche retenue tout en mettant en lumière plusieurs axes d'amélioration, discutés dans la conclusion générale de ce rapport.

Conclusion générale et perspectives

Ce stage, effectué au sein de la Direction des Systèmes d'Information du Ministère de l'Agriculture, de la Pêche Maritime, du Développement Rural et des Eaux et Forêts, m'a permis de concevoir et de développer une application intelligente d'automatisation de la rédaction des notes DSI, répondant à un besoin métier concret identifié en début de mission.

Synthèse du travail réalisé

Le système développé combine plusieurs briques d'intelligence artificielle complémentaires : un modèle de classification d'images fondé sur un réseau de neurones convolutif (ResNet18, par transfert d'apprentissage) pour identifier automatiquement la nature des documents joints aux emails ; un système multi-agents orchestré avec le framework CrewAI, s'appuyant sur un modèle de langage exécuté localement, pour extraire et reformuler les informations pertinentes d'un email en langage administratif ; et un module de génération de documents Word reproduisant fidèlement la charte graphique officielle du Ministère. L'ensemble est exposé à travers une interface web simple, conçue avec Streamlit, qui préserve à chaque étape un contrôle humain sur le résultat produit avant tout téléchargement.

Les résultats obtenus, tant sur le plan quantitatif (exactitude du modèle CNN supérieure à 95 % en validation) que qualitatif (pertinence du contenu généré, fidélité graphique du document final), démontrent la faisabilité et l'intérêt d'une telle solution pour alléger une tâche administrative répétitive.

Apports personnels du stage

Sur le plan technique, ce stage a permis d'approfondir des compétences dans plusieurs domaines complémentaires à la formation Big Data & Intelligence Artificielle suivie à l'UIR : l'entraînement et le transfert d'apprentissage de réseaux de neurones convolutifs avec PyTorch, l'orchestration de systèmes multi-agents à base de modèles de langage, l'intégration à des API tierces (Gmail), ainsi que la génération programmatique de documents bureautiques structurés. Sur le plan méthodologique, ce projet a également

été l’occasion de manipuler un cycle complet de développement logiciel, de l’analyse des besoins jusqu’aux tests, en environnement professionnel réel.

Sur le plan humain, ce stage a permis de découvrir le fonctionnement d’une direction des systèmes d’information au sein d’une administration publique marocaine, ainsi que les enjeux propres à la transformation digitale du service public, notamment en matière de confidentialité des données et d’adoption des outils par les utilisateurs finaux.

Perspectives d’amélioration

Plusieurs axes d’amélioration ont été identifiés à l’issue de ce stage, qui pourraient faire l’objet de développements futurs :

- **Enrichissement du jeu de données** du modèle CNN par des documents réels annotés, afin de réduire l’écart de performance observé entre données synthétiques et documents réels ;
- **Traitement multi-pièces-jointes**, permettant de classer et d’intégrer plusieurs documents joints à un même email dans la note générée ;
- **Historisation des notes générées** dans une base de données, afin de permettre la recherche et le suivi des notes produites au fil du temps ;
- **Extension à d’autres types de notes administratives**, au-delà des seules notes DSI, en généralisant l’architecture à d’autres directions du Ministère ;
- **Déploiement en production** sécurisé, avec gestion multi-utilisateurs et authentification renforcée, actuellement hors du périmètre de ce prototype de stage.

En définitive, ce projet illustre concrètement comment des technologies d’intelligence artificielle – vision par ordinateur et modèles de langage – peuvent être combinées pour automatiser une tâche administrative à faible valeur ajoutée, tout en conservant l’humain au centre du processus de décision, dans une logique d’assistance plutôt que de substitution.